CS430/910: Foundations of Data Analytics

Classification 3 | Dr Ayse Sunar

Objectives

- To understand the concept of classifiers and their construction
- See how decision tree classifiers are constructed
- Statistical classifiers: Naive Bayes
- Linear separation via SVM classifiers
- Construct and compare classifiers using Weka

Part C: Decision Trees

How to Choose Splits

Attribute Selection Measures

- Attribute selection measures provide a ranking for each attribute describing the given training tuples.
- The attribute having the “best” score (highest or lowest depending on the method) is chosen as the splitting attribute for the given tuples.
- Three popular attribute selection measures:

    1. **Information Gain**
    2. **Gain Ratio**
    3. **Gini Index**

Gini Index

- The Gini Index is used by CART (Classification and Regression Trees).
- It measures the impurity of D, as:

$$Gini(D) = 1 - \sum_{i=1}^{m} p_i^2$$

- p_i is the probability that a tuple in D belongs to class C_i, and is estimated by $\frac{|C_{i,D}|}{|D|}$. It is then computed over m classes.

Gini Index

- For each attribute, the Gini Index considers a binary split.
- Consider the case where A is a discrete-valued attribute with v distinct values, $\{a_1, a_2, \ldots, a_v\}$, occurring in D.
- To determine the best binary split, we examine all possible subsets of known values of A.
  - If A has v possible values, then there are 2^v possible subsets.

Gini Index

- Let $module$ have three possible values, $\{CS118, CS430, CS910\}$.
  - The possible subsets are:
    $\{CS118, CS430, CS910\}, \{CS118, CS430\}, \{CS118, CS910\},$
    $\{CS430, CS910\}, \{CS118\}, \{CS430\}, \{CS910\} and, \{\}.$
  - $\{CS118, CS430, CS910\}$ and $\{\}$ (the empty set) are excluded, as they do not represent a split.
    - Hence, there are $\frac{(2^v-2)}{2}$ possible ways to form two partitions of the data D from the binary split on A.

Gini Index

- If a binary split on A partitions D into D_1 and D_2, the Gini Index of D given that partitioning is:

$$Gini_A(D) = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2)$$

- For each attribute, each of the possible binary splits is considered.
- For a discrete-valued, the subset that gives the **minimum** Gini index for that attribute is selected as the splitting subset.

Gini Index

- For continuous-valued attributes, the midpoint between each pair of (sorted) values is taken as a possible split-point.
- The point giving the minimum Gini index for a given (continuous-valued) attribute is taken as the split-point of that attribute.
- Remember, for a given split point of A, D_1 is the set of tuples in D satisfying $A \leq split_point$, and D_2 is the set of tuples in D satisfying $A > split_point$.

Gini Index

- The reduction in impurity incurred by a binary split on a discrete or continuous-valued attribute A is:

$$\Delta Gini(A) = Gini(D) - Gini_A(D)$$

- The attribute that maximises the reduction in impurity is selected as the splitting attribute.
  - This attribute and its splitting subset (or splitting point for a continuous-valued splitting attribute) together form the *splitting criterion*.

Gini Index

Example

- We must first compute $Gini(D)$ as follows:

$$Gini(D) = 1 - \sum_{i=1}^{m} p_i^2$$
$$= 1 - \left(\frac{4}{6}\right)^2 - \left(\frac{2}{6}\right)^2$$
$$= 0.444...$$

- Now, we must compute the Gini Index for each attribute…

ID	income	is_student	Class:plays_sports
1	medium	yes	yes
2	high	no	yes
3	low	yes	no
4	low	yes	yes
5	high	yes	no
6	high	no	yes

Gini Index

Example

- *income* binary subsets are $\{\{low, medium\}, \{high\}\}$, $\{\{low, high\}, \{medium\}\}$, $\{\{medium, high\}, \{low\}\}$.

$$
\begin{aligned}
& Gini_{income \in \{low, medium\}}(D) \\
&= \frac{3}{6} Gini(D_1) + \frac{3}{6} Gini(D_2) \\
&= \frac{3}{6}\left(1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2\right) + \\
&\quad \frac{3}{6}\left(1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2\right) \\
&= 0.444... = Gini_{income \in \{high\}}(D)
\end{aligned}
$$

ID	income	is_student	Class:plays_sports
1	medium	yes	yes
2	high	no	yes
3	low	yes	no
4	low	yes	yes
5	high	yes	no
6	high	no	yes

Gini Index

Example

- *income* binary subsets are $\{\{low, medium\}, \{high\}\}$, $\{\{low, high\}, \{medium\}\}$, $\{\{medium, high\}, \{low\}\}$.

$$
\begin{aligned}
& Gini_{income \in \{low, high\}}(D) \\
& = \frac{5}{6} Gini(D_1) + \frac{1}{6} Gini(D_2) \\
& = \frac{5}{6}\left(1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2\right) + \\
& \quad \frac{1}{6}\left(1 - \left(\frac{1}{1}\right)^2\right) \\
& = 0.4 = Gini_{income \in \{medium\}}(D)
\end{aligned}
$$

ID	income	is_student	Class:plays_sports
1	medium	yes	yes
2	high	no	yes
3	low	yes	no
4	low	yes	yes
5	high	yes	no
6	high	no	yes

Gini Index

Example

- *income* binary subsets are $\{\{low, medium\}, \{high\}\}$, $\{\{low, high\}, \{medium\}\}$, $\{\{medium, high\}, \{low\}\}$.

$$
\begin{aligned}
& Gini_{income \in \{medium, high\}}(D) \\
& = \frac{4}{6} Gini(D_1) + \frac{2}{6} Gini(D_2) \\
& = \frac{4}{6}\left(1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2\right) + \\
& \quad \frac{2}{6}\left(1 - \left(\frac{1}{2}\right)^2 - \left(\frac{1}{2}\right)^2\right) \\
& = 0.41666... = Gini_{income \in \{low\}}(D)
\end{aligned}
$$

ID	income	is_student	Class:plays_sports
1	medium	yes	yes
2	high	no	yes
3	low	yes	no
4	low	yes	yes
5	high	yes	no
6	high	no	yes

Gini Index

Example

- $is_student$ binary subsets are $\{\{yes\}, \{no\}\}$.

$$
\begin{aligned}
& Gini_{is_student \in \{yes\}}(D) \\
& = \frac{4}{6} Gini(D_1) + \frac{2}{6} Gini(D_2) \\
& = \frac{4}{6}\left(1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2\right) + \\
& \quad \frac{2}{6}\left(1 - \left(\frac{2}{2}\right)^2\right) \\
& = 0.333... = Gini_{is_student \in \{no\}}(D)
\end{aligned}
$$

ID	income	is_student	Class:plays_sports
1	medium	yes	yes
2	high	no	yes
3	low	yes	no
4	low	yes	yes
5	high	yes	no
6	high	no	yes

Gini Index

Example

- Summary:

$$Gini_{income \in \{low, medium\}}(D) = Gini_{income \in \{high\}}(D) = 0.444...$$
$$Gini_{income \in \{low, high\}}(D) = Gini_{income \in \{medium\}}(D) = 0.4$$
$$Gini_{income \in \{medium, high\}}(D) = Gini_{income \in \{low\}}(D) = 0.41666...$$
$$Gini_{is_student \in \{yes\}}(D) = Gini_{is_student \in \{no\}}(D) = 0.333...$$

- Therefore, the best binary split on $income$ is $\{low, high\}$ (or $\{medium\}$), as it minimises the Gini index. The best binary split on $is_student$ is $\{yes\}$ (or $\{no\}$), because the attribute itself is binary.

Gini Index

Example

Minimum Gini Index overall.

- Summary:

$$Gini_{income \in \{low, medium\}}(D) = Gini_{income \in \{high\}}(D) = 0.444...$$
$$Gini_{income \in \{low, high\}}(D) = Gini_{income \in \{medium\}}(D) = 0.4$$
$$Gini_{income \in \{medium, high\}}(D) = Gini_{income \in \{low\}}(D) = 0.41666...$$
$$Gini_{is_student \in \{yes\}}(D) = Gini_{is_student \in \{no\}}(D) = 0.333...$$

- Therefore, the best binary split on $income$ is $\{low, high\}$ (or $\{medium\}$), as it minimises the Gini index. The best binary split on $is_student$ is $\{yes\}$ (or $\{no\}$), because the attribute itself is binary.

Gini Index

Example

- Recall:
  - The reduction in impurity incurred by a binary split on a discrete or continuous-valued attribute A is:

$$\Delta Gini(A) = Gini(D) - Gini_A(D)$$

- Therefore:

$$\begin{aligned}\Delta Gini(A) &= Gini(D) - Gini_A(D) \\ &= 0.444... - 0.333... \\ &= 0.111...\end{aligned}$$

- $is_student$ and its splitting subset form the splitting criterion.

Split Rules

- The three measures, in general, return good results, but:
  - **Information Gain**: biased towards multivalued attributes
  - **Gain Ratio**: tends to prefer unbalanced splits in which one partition is much smaller than the others
  - **Gini Index**: bias to multivalued attributes, struggles when number of classes is large
- Many other split rules have been suggested, including:
  - **CHAID**: uses the statistical χ^2 test to choose split
  - **C-SEP**
  - **G-statistic**
  - **Minimum Description Length**: seeks the simplest overall tree
    - Requires fewest bits to describe

Pruning Trees

- Building a tree down to leaves that are pure is likely to overfit
  - **Pure**: only contain members of a single class
- Better results can be obtained by pruning the tree
  - Removing some branches from the tree to reduce size/complexity
- **Prepruning**: add additional stopping conditions when building
  - E.g. limit the depth of the tree (length of path)
  - E.g. stop when the number of examples in a subtree is small
- **Postpruning**: build full tree, then decide which branches to prune
  - Easier to measure difference before/after pruning

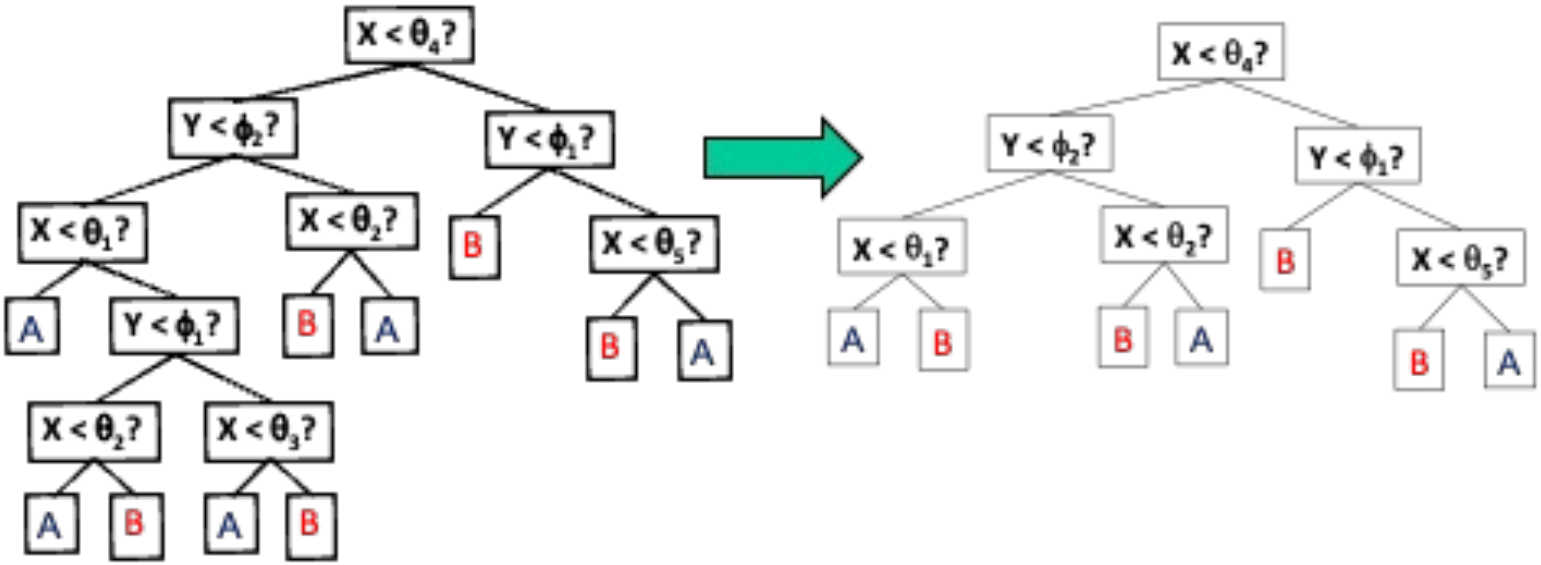
Pruning Trees

More Information

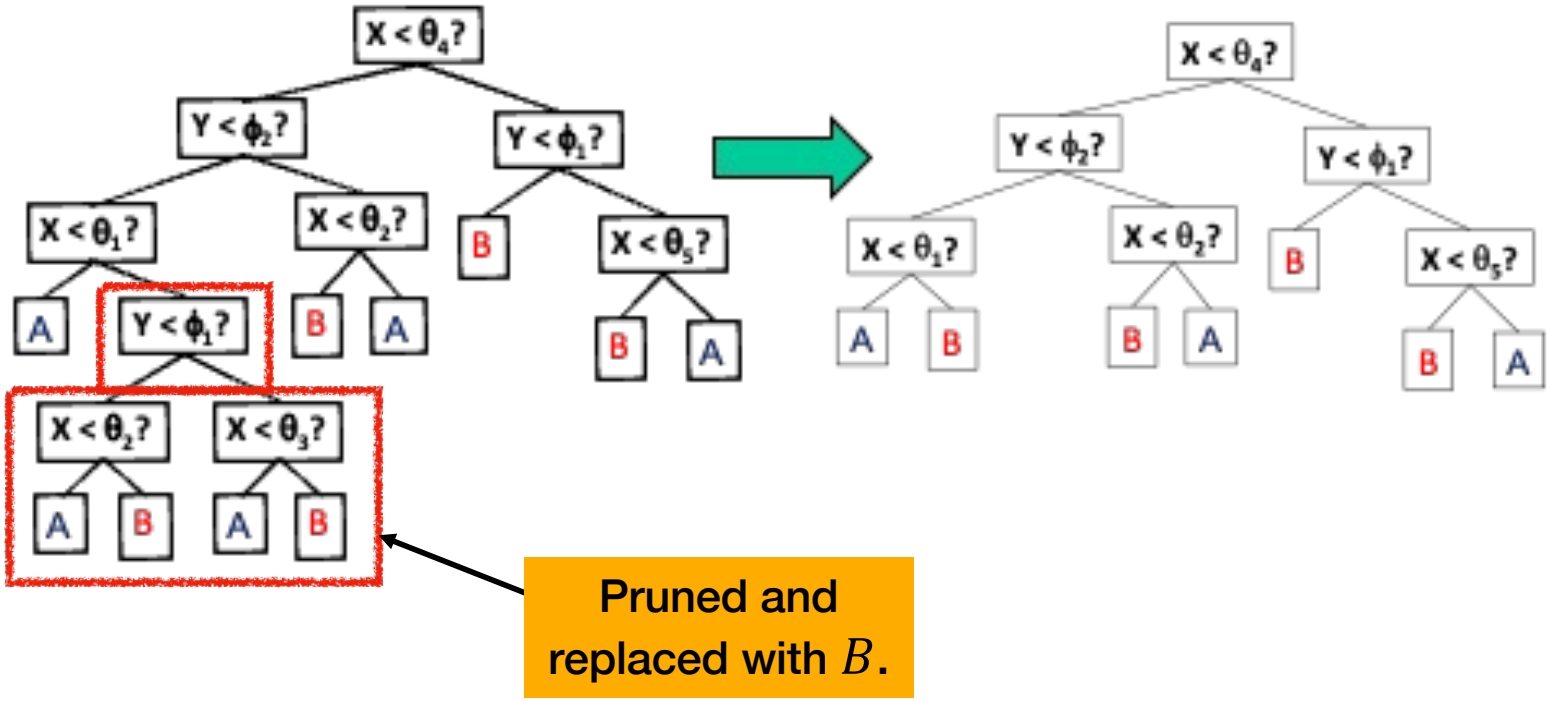
- **Prepruning**: Following application of measure such as Gini Index, Information Gain, etc., if a split falls below a certain threshold, we may instead halt further partitioning.

- **Postpruning**: A subtree at a given node is pruned and replaced with a leaf node. The leaf node is labeled with the most frequent class.

Pruning Trees



Pruning Trees



Pruning Trees

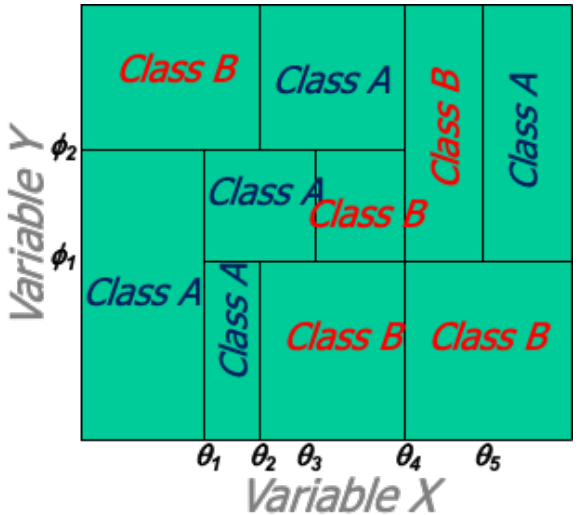
- Prune based on tradeoff between complexity and error
  - Complexity: measure of the complexity of the subtree
  - Error: measured on pruning set (separate from training and test set)
- Reduced error pruning is simple and fast
  - Start at leaves, replace node with majority class
  - Accept if this does not affect accuracy on pruning set

Pruning Trees

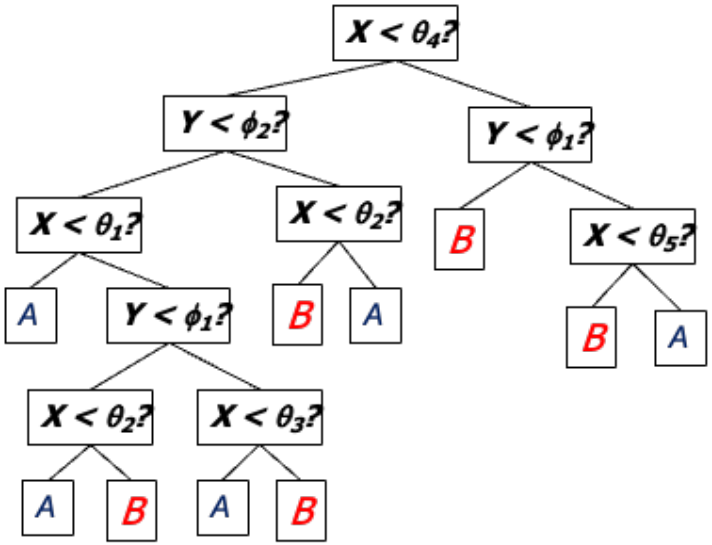
- Cost complexity pruning (CART[1]) uses a more complex function
  - Given current tree T (with $|T|$ leaves) and T' with a node removed
  - Measure E = Error of T on pruning set, E' = error of T' on pruning set
  - Select T' that minimises $\frac{E'-E}{|T|-|T'|}$
  - Can evaluate all generated trees on test set, pick the best

[1]Breiman, L., Friedman, J.H., Olshen, R.A. and Stone, C.J., 2017. *Classification and regression trees*. Routledge.

Classifying with a Decision Tree

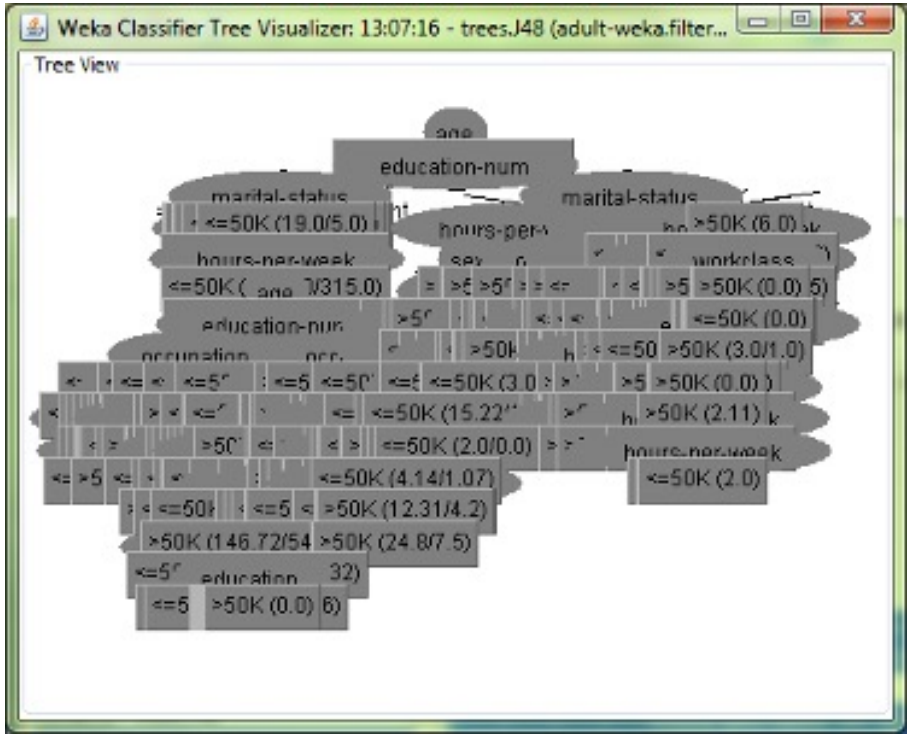


- Classifying a new example with a decision tree is straightforward
  - Start at the root
  - Apply the first test, and determine which branch to follow
  - Iterate until a leaf is reached
  - Apply the label at the leaf



Decision Trees in Weka

- **trees.id3** implements ID3 (Information Gain)
  - Not robust: does not allow missing attributes
- **trees.J48** implements C4.5 (Information Gain scaled by Split Info)



Weka Results for adult.data via J48

```
J48 pruned tree
------------------
age <= 27: <=50K (12012.0/369.0)
age > 27
|   education-num <= 12
|   |   marital-status = Married-civ-spouse
|   |   |   education-num <= 8: <=50K (2323.0/304.0)
|   |   |   education-num > 8
|   |   |   |   hours-per-week <= 35: <=50K (1410.0/315.0)
|   |   |   |   hours-per-week > 35
|   |   |   |   |   age <= 35: <=50K (2760.0/833.0)
|   |   |   |   |   age > 35
|   |   |   |   |   |   education-num <= 9
|   |   |   |   |   |   |   occupation = Tech-support
|   |   |   |   |   |   |   |   workclass = Private: >50K (55.88/23.32)
|   |   |   |   |   |   |   |   workclass = Self-emp-not-inc: <=50K (1.03/0.01)
|   |   |   |   |   |   |   |   workclass = Self-emp-inc: >50K (0.0)
|   |   |   |   |   |   |   |   workclass = Federal-gov: >50K (10.35/3.24)
|   |   |   |   |   |   |   |   workclass = Local-gov: >50K (1.03/0.02)
|   |   |   |   |   |   |   |   workclass = State-gov: <=50K (5.17/1.05)
|   |   |   |   |   |   |   |   workclass = Without-pay: >50K (0.0)
|   |   |   |   |   |   |   |   workclass = Never-worked: >50K (0.0)
```

Weka Results for adult.data via J48

```
Attributes:   age, workclass, education, education-num, marital-status, occupation,
    relationship, race, sex, hours-per-week, native-country, class
Test mode:split 66.0% train, remainder test
Number of Leaves  : 549
Size of the tree :      682
Time taken to build model: 8.4 seconds
=== Evaluation on test split ===
Correctly Classified Instances       13938               83.9335 %
Incorrectly Classified Instances      2668               16.0665 %
Kappa statistic                          0.5337
Mean absolute error                      0.2263
Root mean squared error                  0.3399
Relative absolute error                 62.4059 %
Root relative squared error             80.3307 %
Total Number of Instances            16606
=== Detailed Accuracy By Class ===
               TP Rate   FP Rate   Precision   Recall  F-Measure   ROC Area  Class
                 0.602     0.088      0.675     0.602     0.636      0.865    >50K
                 0.912     0.398      0.883     0.912     0.897      0.865    <=50K
Weighted Avg.    0.839     0.326      0.834     0.839     0.836      0.865

=== Confusion Matrix ===
a     b   <-- classified as
  2335  1543 |     a = >50K
  1125 11603 |     b = <=50K
```

Part D: Naïve Bayes & SVM

Bayesian Classification Methods

- Bayesian classifiers are statistical classifiers based on Bayes' theorem.
  - Predicts the probability that a given tuple belongs to a particular class.
- An example of a simple Bayesian classifier is the Naïve Bayesian classifier:
  - Assumes that the effect of an attribute value on a given class is independent of the values of the other attributes. This assumption is called *class-conditional independence*.

Bayes Theorem

- Let $\mathbf{X}$ be a data tuple, which in Bayesian terms is considered *evidence*, described by measurements made on a set of n attributes.
- Let H be some hypothesis, such as that data tuple $\mathbf{X}$ belongs to class C.
- For classification problems, we want to determine $P(H|\mathbf{X})$ - the probability that the hypothesis H holds, given the evidence $\mathbf{X}$.
  - Or, the probability that tuple $\mathbf{X}$ belongs to class C, given that we know the attribute description of $\mathbf{X}$.

Bayes Theorem

- $P(H|\mathbf{X})$ is the *posterior probability* (or *a posteriori probability*) of H conditioned on $\mathbf{X}$.
  - Example: Let our data tuples describe people by area and income, and that $\mathbf{X}$ is a person living in Coventry with an income of £25,000. Let H be the hypothesis that the person will attend *Folklorica* at the Warwick Arts Centre. $P(H|\mathbf{X})$ reflects the probability that person $\mathbf{X}$ will attend *Folklorica* given that we know their area and income.

Bayes Theorem

- $P(H)$ is the *prior probability* (or *a priori probability*) of H.
  - From the previous example, $P(H)$ is the probability that any person will attend *Folklorica*, regardless of location or income.
- $P(\mathbf{X}|H)$ is the posterior probability of $\mathbf{X}$ conditioned on H.
  - From the previous example, $P(\mathbf{X}|H)$ is the probability that a person, $\mathbf{X}$, lives in Coventry and has an income of £25,000, given that we know that they will attend *Folklorica*.

Bayes Theorem

- $P(\mathbf{X})$ is the *prior probability* (or *a priori probability*) of $\mathbf{X}$.
  - From the previous example, $P(\mathbf{X})$ is the probability that our person lives in Coventry and earns £25,000.

- $P(H)$, $P(\mathbf{X}|H)$ and $P(\mathbf{X})$ can be estimated from the given data, $P(H|\mathbf{X})$ can be created using Bayes' Theorem:

$$P(H|\mathbf{X}) = \frac{P(\mathbf{X}|H)P(H)}{P(\mathbf{X})}$$

Naïve Bayes Classifier

- Also known as the *Simple Bayesian classifier*.

1. Let D be a training set of tuples and their associated class labels. Each tuple is an n-dimensional attribute vector $\mathbf{X} = (x_1, x_2, \ldots, x_n)$, depicting n measurements from n attributes $(A_1, A_2, \ldots, A_n)$.

2. Let there be m classes $(C_1, C_2, \ldots, C_m)$. For a tuple $\mathbf{X}$, the classifier will predict that $\mathbf{X}$ belongs to the class having the highest posterior probability, conditioned on $\mathbf{X}$, i.e. the classifier will predict that $\mathbf{X}$ belongs to class C_i if:

$$P(C_i|\mathbf{X}) > P(C_j|\mathbf{X}) for 1 \leq j \leq m, j \neq i$$

Naïve Bayes Classifier

- Reminder, we can calculate $P(C_i|\mathbf{X})$ by:

$$P(C_i|\mathbf{X}) = \frac{P(\mathbf{X}|C_i)P(C_i)}{P(\mathbf{X})}$$

- The class C_i for which $P(C_i|\mathbf{X})$ is maximised, is called the *maximum posteriori hypothesis*.

Naïve Bayes Classifier

- Reminder, we can calculate $P(C_i|\mathbf{X})$ by:

$$P(C_i|\mathbf{X}) = \frac{P(\mathbf{X}|C_i)P(C_i)}{\boxed{P(\mathbf{X})}}$$

Constant for all classes.

- The class C_i for which $P(C_i|\mathbf{X})$ is maximised, is called the *maximum posteriori hypothesis*.

Naïve Bayes Classifier

3. As $P(\mathbf{X})$ is constant for all classes, only $P(\mathbf{X}|C_i)P(C_i)$ needs to be maximised.

   - If the class prior probabilities ($P(C_i)$) are not known, then we commonly assume that all classes are equally likely (i.e. $P(C_1) = P(C_2) =\ldots = P(C_m)$).=, meaning we would only need to maximise $P(\mathbf{X}|C_i)$.

   - Class prior probabilities can be estimated by $P(C_i) = \frac{|C_{i,D}|}{|D|}$, where $|C_{i,D}|$ is the number of training tuples of class C_i in D.

Naïve Bayes Classifier

4. Due to the naive assumption of class-conditional independence (which presumes that the attributes' values are conditionally independent of one another, given the class label of the tuple). We can calculate $P(\mathbf{X} \mid C_i)$ as follows:

$$P(\mathbf{X} \mid C_i) = \prod_{k=1}^{n} P(x_k \mid C_i)$$
$$= P(x_1 \mid C_i) \times P(x_2 \mid C_i) \times \ldots \times P(x_n \mid C_i)$$

- Assuming A_k is categorical, $P(x_k \mid C_i)$ is the number of tuples of class C_i in D having the value x_k for A_k, divided by $|C_{i,D}|$ (the number of tuples of class C_i in D).

Naïve Bayes Classifier

5. We now compute $P(\mathbf{X}|C_i)P(C_i)$ for each class C_i. The classifier predicts that the class label of tuple $\mathbf{X}$ is the class C_i if and only if:

$$P(\mathbf{X}|C_i)P(C_i) > P(\mathbf{X}|C_j)P(C_j) for 1 \leq j \leq m, j \neq i$$

i.e. the predicted class label is the class C_i for which $P(\mathbf{X}|C_i)P(C_i)$ is the maximum.

Naïve Bayes Classifier

- A further point to raise…
  - Consider the situation where $P(x_k \mid C_i) = 0$ in the training set (e.g., from our previous example $P(area = Coventry \mid visits_Folklorica = yes) = 0$).
  - Recall the following equation:

$$
\begin{aligned}
P(\mathbf{X} \mid C_i) &= \prod_{k=1}^{n} P(x_k \mid C_i) \\
&= P(x_1 \mid C_i) \times P(x_2 \mid C_i) \times \ldots \times P(x_n \mid C_i)
\end{aligned}
$$

Naïve Bayes Classifier

- A further point to raise…
  - Consider the situation where $P(x_k \mid C_i) = 0$ in the training set (e.g., from our previous example $P(area = Coventry \mid visits_Folklorica = yes) = 0$).
  - Recall the following equation:

If this is 0…

$$\boxed{P(\mathbf{X} \mid C_i)} = \prod_{k=1}^{n} P(x_k \mid C_i)$$

$$= \boxed{P(x_1 \mid C_i)} \times P(x_2 \mid C_i) \times \ldots \times P(x_n \mid C_i)$$

…so is this!

Naïve Bayes Classifier

- Simple solution:
  - We assume our training data, D, is so large that adding one to each count would make a negligible difference, yet would avoid the problem on the previous slide.
    - e.g. add one tuple where a person from Coventry visits Folklorica.
    - $P(area = Coventry | visits_Folklorica = yes)$ is no longer 0, but is very small!
  - This process is known as *Laplacian Correction* (more info in book).

Naïve Bayes Classifier

Example

age	income	student	credit_rating	buys_computer
youth	high	no	fair	no
youth	high	no	excellent	no
middle_aged	high	no	fair	yes
senior	medium	no	fair	yes
senior	low	yes	fair	yes
senior	low	yes	excellent	no
middle_aged	low	yes	excellent	yes
youth	medium	no	fair	no
youth	low	yes	fair	yes
senior	medium	yes	fair	yes
youth	medium	yes	excellent	yes
middle_aged	medium	no	excellent	yes
middle_aged	high	yes	fair	yes
senior	low	no	excellent	no

Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.

Naïve Bayes Classifier

$$P(C_i|\mathbf{X}) = \frac{P(\mathbf{X}|C_i)P(C_i)}{P(\mathbf{X})}$$

Example

- Let's say we want to classify
  $X = (age = youth, income = medium, student = yes, credit_rating = fair)$
- Let's first calculate $P(C_i)$ for all classes:
  - $P(buys_computer = yes) = \frac{9}{14} = 0.643$
  - $P(buys_computer = no) = \frac{5}{14} = 0.357$

Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.

Naïve Bayes Classifier

$$P(C_i|\mathbf{X}) = \frac{P(\mathbf{X}|C_i)P(C_i)}{P(\mathbf{X})}$$

Example

- To compute $P(\mathbf{X}|C_i), for i = 1,2$, we need:
  - $P(age = youth|buys_computer = yes) = \frac{2}{9} = 0.222$
  - $P(age = youth|buys_computer = no) = \frac{3}{5} = 0.6$
  - $P(income = medium|buys_computer = yes) = \frac{4}{9} = 0.444$
  - $P(income = medium|buys_computer = no) = \frac{2}{5} = 0.400$

Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.

Naïve Bayes Classifier

$$P(C_i|\mathbf{X}) = \frac{P(\mathbf{X}|C_i)P(C_i)}{P(\mathbf{X})}$$

Example

- To compute $P(\mathbf{X}|C_i), for i = 1,2$, we need:
  - $P(student = yes|buys_computer = yes) = \frac{6}{9} = 0.667$
  - $P(student = yes|buys_computer = no) = \frac{1}{5} = 0.200$
  - $P(credit_rating = fair|buys_computer = yes) = \frac{6}{9} = 0.667$
  - $P(credit_rating = fair|buys_computer = no) = \frac{2}{5} = 0.400$

Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.

Naïve Bayes Classifier

$$P(C_i|\mathbf{X}) = \frac{P(\mathbf{X}|C_i)P(C_i)}{P(\mathbf{X})}$$

Example

- Next, compute $P(\mathbf{X}|buys_computer = yes)$:

$$\begin{aligned} P(\mathbf{X}|buys_computer = yes) &= P(age = youth|buys_computer = yes) \\ &\times P(income = medium|buys_computer = yes) \\ &\times P(student = yes|buys_computer = yes) \\ &\times P(credit_rating = fair|buys_computer = yes) \\ &= 0.222\times0.444\times0.667\times0.667 = 0.044 \end{aligned}$$

$$P(\mathbf{X}|buys_computer = no) = 0.600\times0.400\times0.200\times0.400 = 0.019$$

Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.

Naïve Bayes Classifier

$$P(C_i|\mathbf{X}) = \frac{P(\mathbf{X}|C_i)P(C_i)}{P(\mathbf{X})}$$

Example

- Finally, find the class which maximises $P(\mathbf{X}|C_i)P(C_i)$:

$P(\mathbf{X}|buys_computer = yes)P(buys_compter = yes) = 0.044 \times 0.643 = 0.028$

$P(\mathbf{X}|buys_computer = no)P(buys_compter = no) = 0.019 \times 0.357 = 0.007$

- Therefore, the classifier predicts $buys_computer = yes$ for tuple $\mathbf{X}$.

Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.

Naïve Bayes in Weka

```
Attribute                          >50K    <=50K
                                 (0.24)   (0.76)
=================================================
age
  mean                           44.2752 36.8722
  std. dev.                      10.5585 14.1039
  weight sum                       11687   37155
  precision                            1       1

workclass
  Private                         7388.0 26520.0
  Self-emp-not-inc                1078.0  2786.0
  Self-emp-inc                     939.0   758.0
  Federal-gov                      562.0   872.0
  Local-gov                        928.0  2210.0
  State-gov                        531.0  1452.0
  Without-pay                        3.0    20.0
  Never-worked                       1.0    11.0
  [total]                        11430.0 34629.0
…
Time taken to build model: 0.51 seconds
```

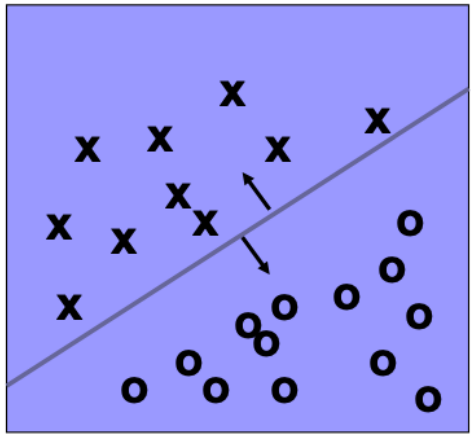
Note: Weka applies the Laplacian Correction to categorical attributes.

Naïve Bayes in Weka

```
Scheme:weka.classifiers.bayes.NaiveBayes
Relation:     adult-weka.filters.unsupervised.attribute.Remove-R3,11-12
Instances:    48842
Test mode:split 66.0% train, remainder test
=== Evaluation on test split ===
Correctly Classified Instances       13519               81.4103 %
Incorrectly Classified Instances      3087               18.5897 %
Kappa statistic                          0.5295
Mean absolute error                      0.2043
Root mean squared error                  0.3681
Relative absolute error                 56.3263 %
Root relative squared error             86.9948 %
Total Number of Instances            16606
=== Detailed Accuracy By Class ===
               TP Rate   FP Rate   Precision   Recall  F-Measure   ROC Area  Class
                 0.751     0.167      0.579     0.751     0.654      0.885    >50K
                 0.833     0.249      0.917     0.833     0.873      0.885    <=50K
Weighted Avg.    0.814     0.23       0.838     0.814     0.822      0.885
=== Confusion Matrix ===
    a     b   <-- classified as
  2913   965 |     a = >50K
  2122 10606 |     b = <=50K
```

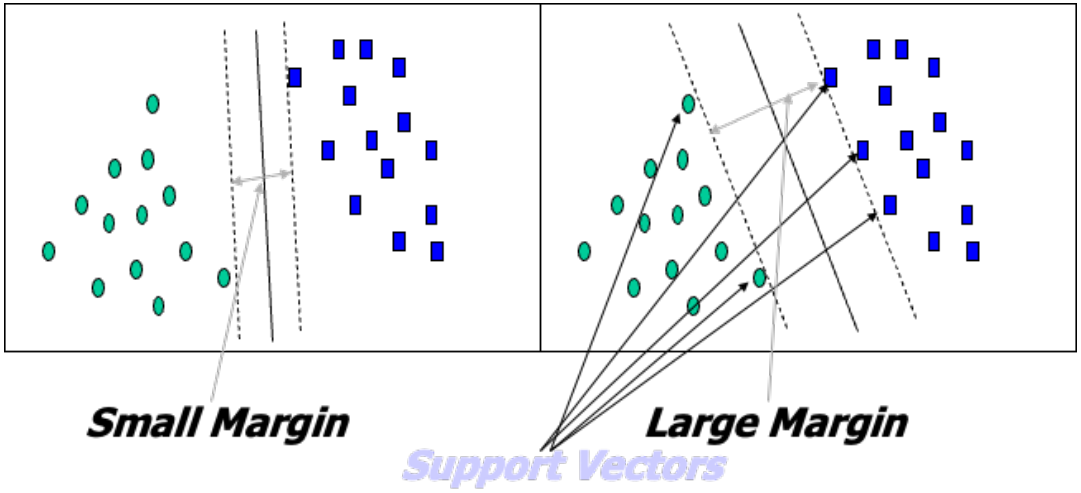
Linearly Separable Data

- Models seen so far have tried to partition up the data space
  - **Decision Trees**: partition into (hyper)rectangles (generalisation to higher dimension)
- Sometimes the data is *linearly separable*
  - Examples of one class fall one side of a line / hyperplane
  - Other class is on other side of the plane
- *Linear classifiers* aim to find 'best' plane
  - Allows a simple description of the model
    - Defined by the separating hyperplane
  - Early examples: *winnow algorithm*, *perceptron* (used in neural networks)



Support Vector Machines (SVM)

- Support Vector Machines (SVM) seek the best plane.
  - When the data is linearly separable, many hyperplanes can separate.
  - SVM seeks the plane that maximises the margin between the classes.



Support Vector Machines (SVM)

1. SVM begins by transforming the original training data into a higher dimension.

2. Once the data has been transformed, SVM searches for a hyperplane/ decision boundary which separates the tuples of one class from another.

Support Vector Machines (SVM)

As an Optimisation Problem

- A hyperplane is written as $\mathbf{W} \cdot \mathbf{X} + b = 0$
  - $\mathbf{W}$: (d-dimensional) weight vector, where d is the number of attributes
  - $\mathbf{X}$: points in (d-dimensional) space
  - b: bias
- Any $\mathbf{X}$ "above" hyperplane has $\mathbf{W} \cdot \mathbf{X} + b > 0$
  - $\mathbf{W} \cdot \mathbf{X} + b < 0$: $\mathbf{X}$ is "below" the hyperplane
- Find weights $\mathbf{W}$ so that
  - $\mathbf{W} \cdot \mathbf{X} + b \geq 1$ for $y = +1$
  - $\mathbf{W} \cdot \mathbf{X} + b \leq -1$ for $y = -1$
- Combining the above gives us: $y(\mathbf{W} \cdot \mathbf{X} + b) \geq 1$ for all (y, X)

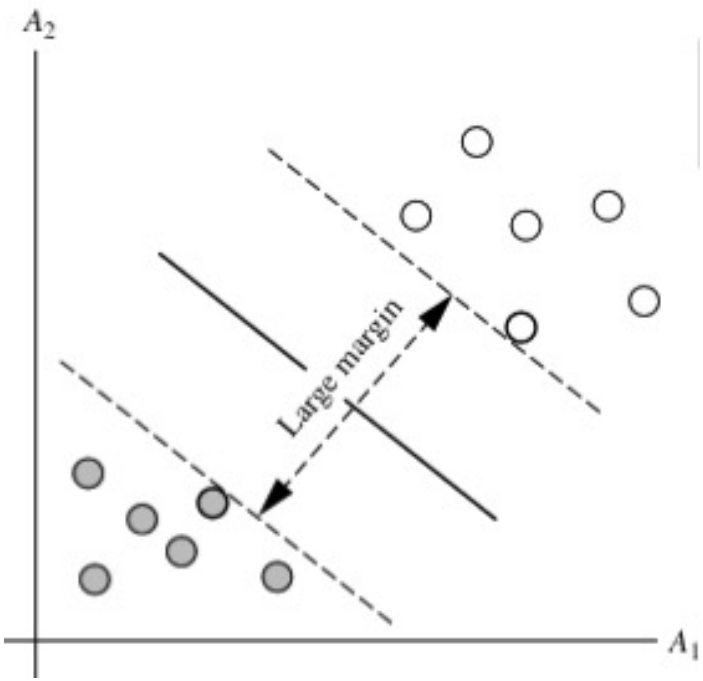


Figure from: Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.

Support Vector Machines (SVM)

As an Optimisation Problem

- Training tuples that fall on the hyperplanes (lets call them H_1 and H_2) are known as *Support Vectors*.
- The distance of a point on H_1 (or H_2) from the plane is $\frac{1}{\|\mathbf{W}\|}$

Euclidean norm of **W**.

- Hence, the maximal margin is $\frac{2}{\|W\|}$.

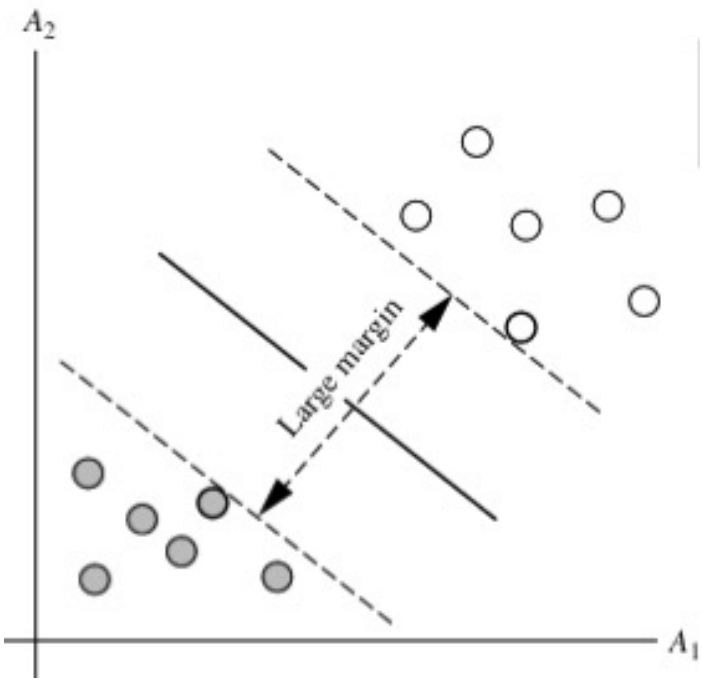


Figure from: Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.

Support Vector Machines (SVM)

As an Optimisation Problem

- This then becomes an optimisation problem…
- We want to maximise $\frac{2}{\|W\|}$.
  - Subject to $y(\mathbf{W} \cdot \mathbf{X} + b) \geq 1$, for all (y, X)

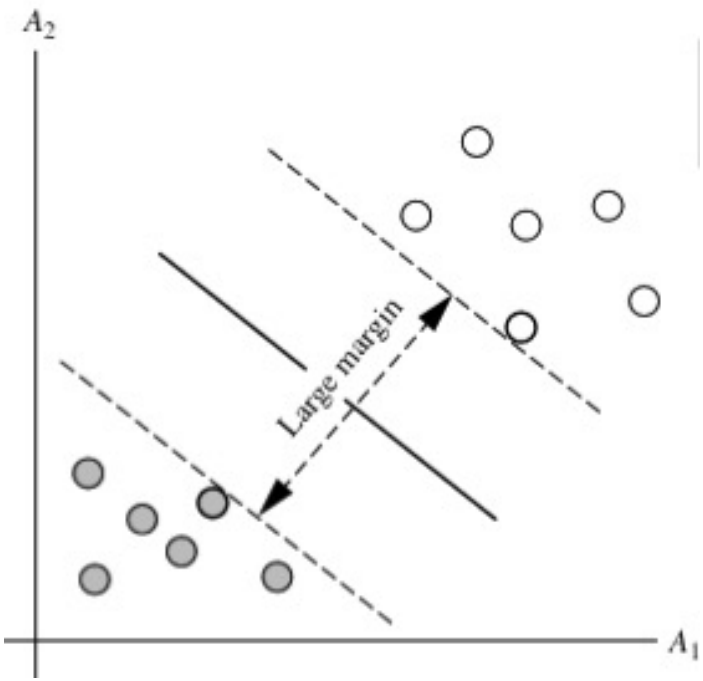


Figure from: Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.

Support Vector Machines (SVM)

Advantages

- Advantages:
  - Given $\mathbf{W}$ and b, SVM is very fast to apply to new examples.
    - If $\mathbf{W} \cdot \mathbf{X} + b > 0$, output $+1$, else output -1
  - Accuracy is generally good
    - Margin determined by only the few support vectors.
  - Overfitting is avoided

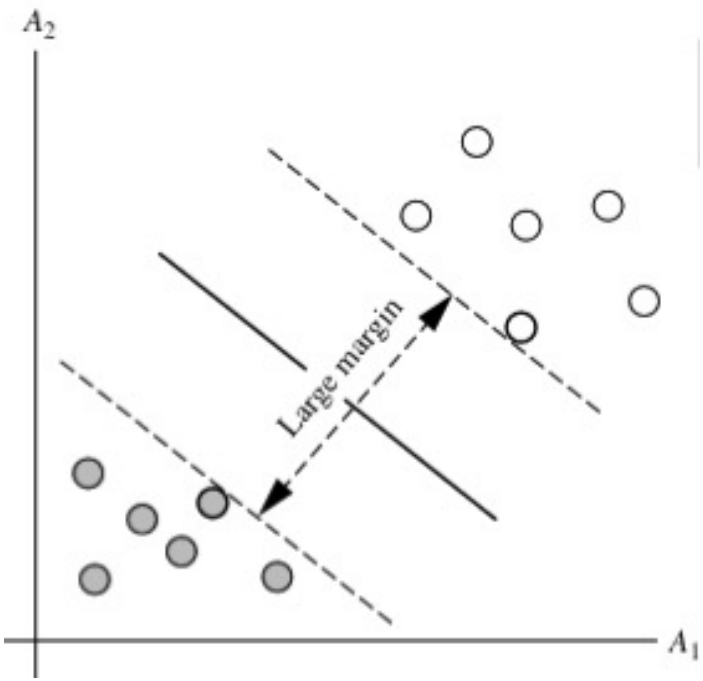


Figure from: Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.

Support Vector Machines (SVM)

Disadvantages

- Disadvantages:
  - Whilst SVM is fast to apply, it is slow to train!
  - Defined only for two classes (need e.g. one-vs.-all for more classes)
  - Assumes data is linearly separable, but this is not true in general.

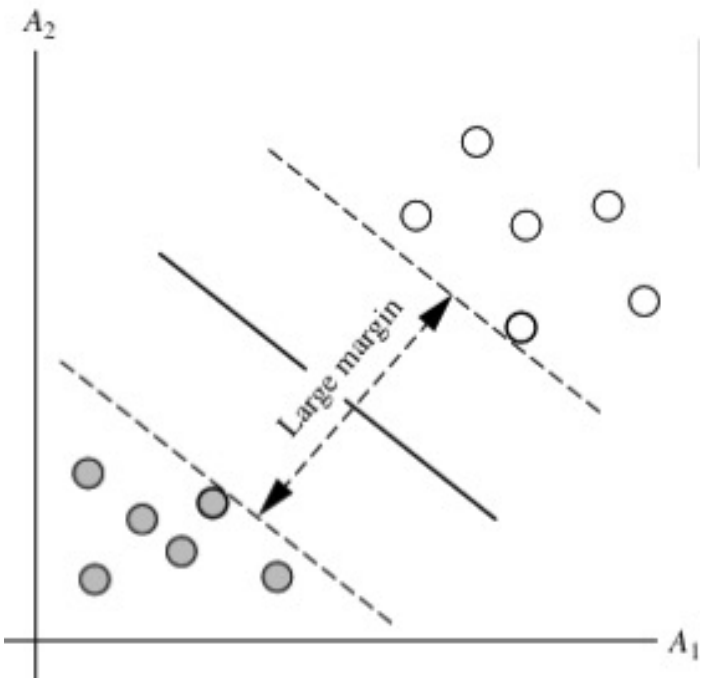


Figure from: Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.

Support Vector Machines (SVM)

Evaluation on adult.data in Weka

```
Scheme:weka.classifiers.functions.SMO -C 1.0 -L 0.001 -P 1.0E-12 -N 0 -V -1 -W 1 -K
    "weka.classifiers.functions.supportVector.PolyKernel -C 250007 -E 1.0"
Relation:     adult-weka.filters.unsupervised.attribute.Remove-R3,11-12
Instances:    48842
Attributes:   12
Test mode:split 66.0% train, remainder test
=== Classifier model (full training set) ===
Kernel used:  Linear Kernel: K(x,y) = <x,y>
Classifier for classes: >50K, <=50K
Time taken to build model: 3472.87 seconds
=== Evaluation on test split ===
Correctly Classified Instances       13912               83.7769 %
Incorrectly Classified Instances      2694               16.2231 %
Total Number of Instances            16606
=== Detailed Accuracy By Class ===
               TP Rate   FP Rate   Precision   Recall  F-Measure   ROC Area  Class
                 0.556     0.076      0.689     0.556     0.615      0.74     >50K
                 0.924     0.444      0.872     0.924     0.897      0.74     <=50K
Weighted Avg.    0.838     0.358      0.829     0.838     0.831      0.74
=== Confusion Matrix ===
     a     b   <-- classified as
  2155  1723 |     a = >50K
   971 11757 |     b = <=50K
```

Support Vector Machines (SVM)

More Information

- We’ve only scratched the surface of SVM…
- Find out more in **CS429/CS909 (Data Mining)** and **CS342 (Machine Learning)**.

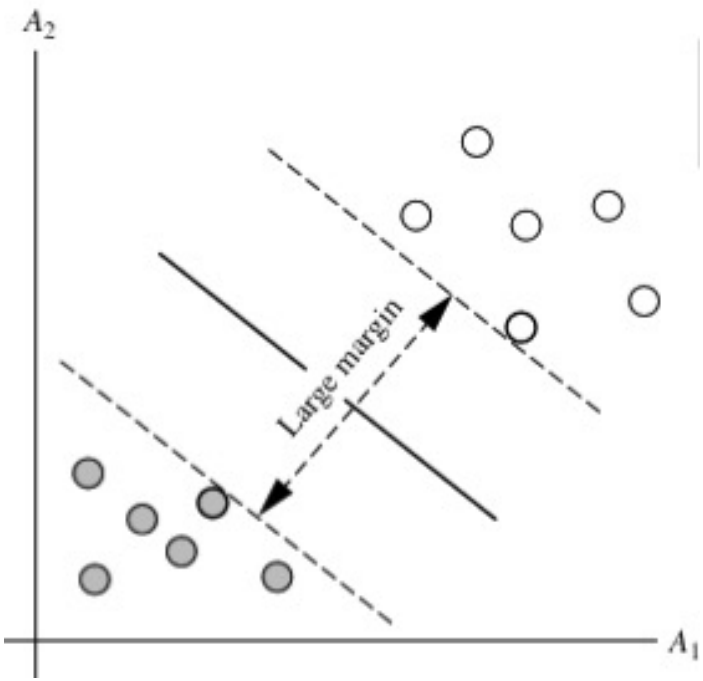


Figure from: Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.

Comparisons of Classifiers on adult.data

Name	Accuracy	AUC	FP Rate	Time to build	Kappa Statistic
Majority Class	0.766	0.5	0.766	0.1s	0
kNearest Neighbour	0.834	0.874	0.299	0.1s	0.5359
Decision tree (J48)	0.839	0.865	0.326	8.4s	0.5377
Naïve Bayes	0.814	0.885	0.23	0.5s	0.5295
SVM	0.837	0.74	0.358	3472s	0.5141
Logistic Regression	0.837	0.888	0.35	31s	0.5175

- Tradeoffs between accuracy, FP rate, AUC, time to build, time to apply, and interpretability of resulting classifier
- Different data sets (or different objectives) will show different behaviour

Summary of Classifiers

- Many different ways to construct classifiers to predict values.
- **General framework**: build on training data, evaluate on test data.
- **Decision tree** classifiers constructed by recursively splitting data.
- **Naïve Bayes** makes independence assumption between variables.
- **SVM** classifiers find linear separation.
- Can construct and compare classifiers using Weka.

Acknowledgements

- Han, J., Pei, J. and Kamber, M., 2012. *Data Mining: Concepts and Techniques*. Elsevier.
- G Cormode [Warwick, CS910]
- F Ciucu [Warwick, CS430/CS910]